

Universitatea Politehnica din București
Facultatea de Automatică și Calculatoare
Seria CC

Tema 1 – Introducere în Machine Learning

Documentație: analiză, preprocesare, regresie și clasificare

Student: Potop Horia-Ioan
Grupa: 333AA
An: 2026

Cuprins

1	Introducere și obiective	2
2	Descrierea setului de date	2
3	Analiza exploratorie a datelor	2
3.1	Variabile numerice	2
3.2	Valori lipsă și variabile categorice	3
3.3	Distribuția țintelor	5
3.4	Corelații și asocieri	5
4	Preprocesare	6
4.1	Decizii comune	6
4.2	Preprocesarea pentru regresie	6
4.3	Preprocesarea pentru clasificare	7
5	Regresie: baseline și perfecționare	7
5.1	Baseline obligatoriu: regresie liniară	7
5.2	Adversarial validation	8
5.3	Etapele de perfecționare pentru regresie	9
5.4	Modelul final de regresie	10
6	Clasificare: baseline și perfecționare	10
6.1	Baseline obligatoriu: Decision Tree	10
6.2	EDA specifică pentru clasificare	11
6.3	Modele încercate pentru perfecționare	11
6.4	Evaluarea finală pentru clasificare	12
7	Rezultate Kaggle și fișiere finale	14
8	Concluzii	14

1 Introducere și obiective

Tema are două direcții principale: prezicerea salariului, ca problemă de regresie, și prezicerea clasei `vacation`, ca problemă de clasificare multi-clasă. Documentația nu este doar o listă de modele rulate, ci o justificare a pașilor care au dus de la soluții simple, ușor de verificat, la modele mai robuste pentru distribuția din testul privat.

Cerințele urmărite în raport sunt: analiză exploratorie pentru variabile numerice și categorice, tratarea valorilor lipsă și a valorilor extreme, baseline-uri obligatorii pentru regresie și clasificare, evaluări cu metricile cerute, comparații între modele și un jurnal al încercărilor relevante. Pentru arhiva de surse, notebook-ul `main.ipynb` păstrează codul cu care au fost generate tabelele, graficele, modelele și submișiile.

2 Descrierea setului de date

Setul de antrenare conține 64.000 de exemple, setul local de test/validare conține 16.000 de exemple, iar testul privat Kaggle conține 16.000 de exemple fără coloanele țintă. În notebook, datele au fost păstrate în aceeași structură pentru cele două task-uri, dar cu ținte și reguli de evitare a leakage-ului diferite.

Tabelul 1: Structura generală a datelor folosite în temă

Coloană / grup	Tip	Rol în analiză
<code>experience_years</code> , <code>skills_count</code> , <code>certifications</code> , <code>total_days_worked</code> , <code>aggregated_score</code>	numeric	Variabile explicative pentru ambele probleme; unele au distribuții cu cozi lungi sau valori speciale.
<code>job_title</code> , <code>company_size</code> , <code>education_level</code> , <code>industry</code> , <code>remote_work</code> , <code>location</code>	categoric	Variabile care descriu contextul profesional; utile mai ales în combinații și în encodări.
<code>salary</code>	numeric	Ținta pentru regresie; nu se folosește ca predictor în clasificare.
<code>vacation</code>	categoric	Ținta pentru clasificare; nu se folosește ca predictor în regresie.

3 Analiza exploratorie a datelor

3.1 Variabile numerice

Primul pas a fost inspectarea statisticilor descriptive pentru coloanele numerice. Scopul nu a fost doar detectarea valorilor extreme, ci și identificarea variabilelor care pot induce overfit sau care pot fi redundante.

Tabelul 2: Statistici descriptive pentru principalele variabile numerice din train

Variabilă	count	mean	std	min	Q1	mediană	Q3	max
experience_years	64000	9.97	6.05	0.00	5.00	10.00	15.00	20.00
skills_count	64000	11.60	10.74	1.00	5.00	10.00	15.00	69.00
certifications	64000	2.51	1.71	0.00	1.00	3.00	4.00	5.00
salary	64000	145683.57	37384.76	31867.00	119209.75	143299.00	169582.25	333046.00
total_days_worked	64000	2393.51	1452.04	-1.00	1200.00	2400.00	3600.00	4800.00
aggregated_score	64000	0.00	1.00	-4.80	-0.67	0.00	0.68	4.22

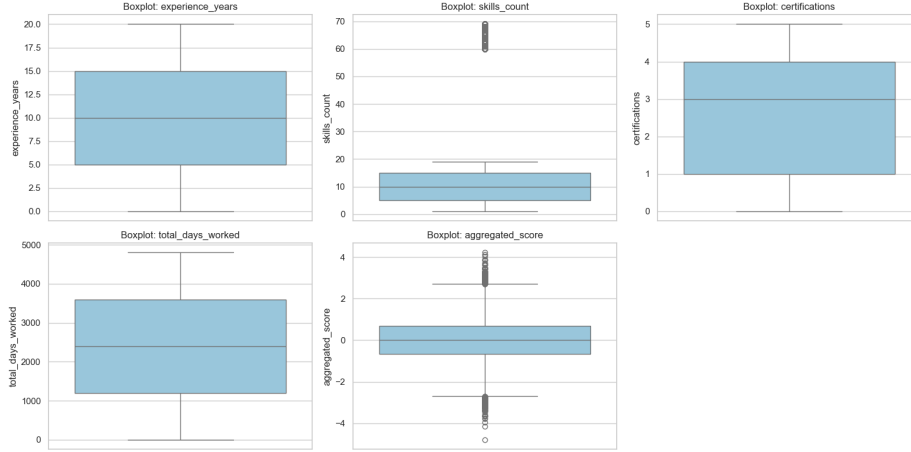


Figura 1: Boxplot-uri pentru variabilele numerice principale. Se observă coada lungă pentru `skills_count`, valorile extreme din `aggregated_score` și valoarea specială `-1` din `total_days_worked`.

Observațiile importante au fost următoarele. `skills_count` are o coadă lungă, cu valori de până la 69, deși 75% dintre exemple au cel mult 15 skill-uri. Pentru modelele liniare, astfel de valori extreme pot deplasa coeficienții și pot crește eroarea pe exemple obișnuite; pentru modelele de arbori, ele pot crea split-uri prea specifice. De aceea, în perfecționare am testat binning și indicatori de outlier, nu doar păstrarea valorii brute. `aggregated_score` este standardizat în jurul lui 0, dar are extreme suficient de mari încât să merite analizat separat; faptul că Pearson față de salariu este aproape zero nu înseamnă că nu există semnal, ci că semnalul nu este liniar. `total_days_worked` este aproape echivalent cu experiența exprimată în zile și include valoarea specială `-1`, deci a fost tratat cu atenție în modelele finale și interpretat ca posibilă redundanță față de `experience_years`.

3.2 Valori lipsă și variabile categorice

Singura lipsă majoră a fost la `remote_work`: 19.153 valori lipsă în train. Această lipsă nu a fost tratată ca zgomot pur, deoarece absența informației poate fi ea însăși semnal: un rând fără informație despre remote poate proveni dintr-un tip diferit de companie sau dintr-un proces diferit de colectare. De aceea, pentru modelele de arbori a fost păstrată categoria `Missing`, iar pentru unele experimente s-a adăugat și indicatorul `remote_work_missing`. Pentru modelele liniare am folosit imputare/one-hot encoding, deoarece acestea nu pot lucra direct cu valori categorice lipsă.



Figura 2: Distribuțiile variabilelor categorice. Majoritatea categoriilor sunt relativ echilibrate, iar **remote_work** are o componentă importantă de valori lipsă.

3.3 Distribuția țințelor

Pentru regresie, **salary** are o distribuție continuă cu valori între aproximativ 31.867 și 333.046. Distribuția are o coadă spre salarii mari, motiv pentru care transformarea **log1p** a fost utilă în modelele avansate: penalizează mai puțin disproporționat valorile mari și ajută modelul să învețe relații multiplicative. Pentru clasificare, **vacation** este dezechilibrată: clasa **No Vacation** domină, iar clasele **Large**, **Medium** și **Small** sunt mai rare. Acest dezechilibru explică de ce am raportat și F1 macro/per-clasă, nu doar accuracy; altfel un model poate părea bun doar pentru că prezice bine clasa majoritară.

Tabelul 3: Distribuția claselor pentru **vacation**

Clasă	Număr exemple	Procent
No Vacation	34686	54.20%
Small	12877	20.12%
Medium	8527	13.32%
Large	7910	12.36%

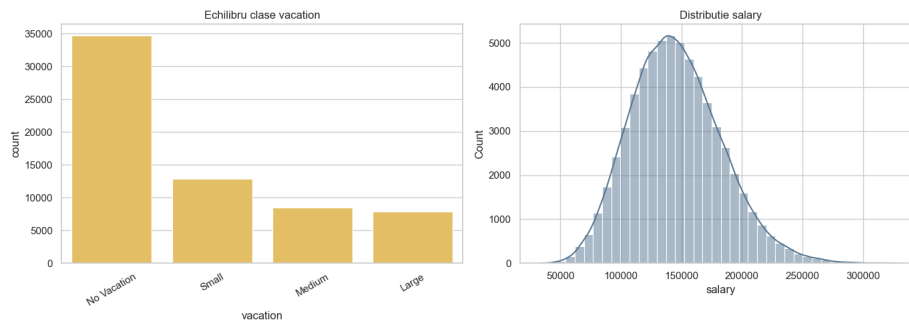


Figura 3: Distribuția țințelor: clasele **vacation** sunt dezechilibrate, iar **salary** are o distribuție unimodală cu o coadă spre salarii mari.

3.4 Corelații și asocieri

Pentru regresie, cele mai puternice relații numerice cu salariul apar la **experience_years** și **total_days_worked**. Cele două au practic același semnal, ceea ce explică de ce, în modelele finale, **total_days_worked** a fost privit mai degrabă ca variabilă redundantă sau de verificare.

Tabelul 4: Corelații Pearson cu **salary**

Variabilă	Corelație cu salary
experience_years	0.437
total_days_worked	0.437
certifications	0.077
skills_count	0.065
aggregated_score	-0.004

Pentru clasificare, analiza nu s-a limitat la corelații numerice. Pentru variabilele cat-

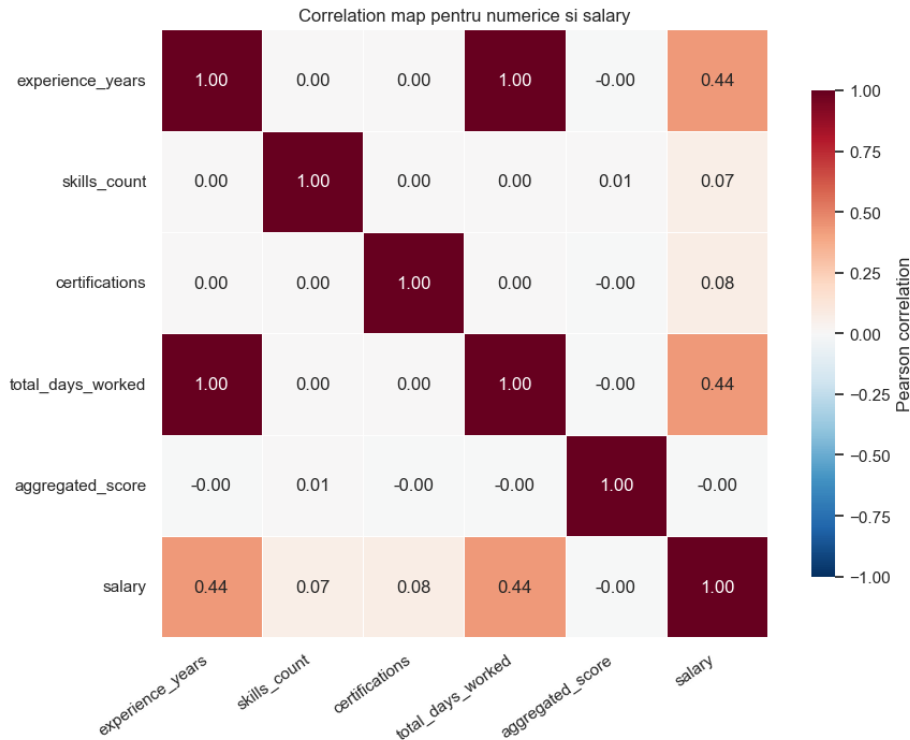


Figura 4: Heatmap Pearson pentru variabilele numerice și salary. `experience_years` și `total_days_worked` sunt aproape redundante și au cea mai clară relație liniară cu salariul.

egorice s-au folosit asocieri de tip Cramer's V / chi-square, deoarece ținta `vacation` este categorică. Această analiză este mai potrivită decât Pearson pentru categorice, fiindcă măsoară asocierea dintre distribuții de clase. Dacă o variabilă are asociere mică individual, ea poate rămâne totuși utilă în combinații, de exemplu rol + locație sau educație + rol. Din acest motiv, la regresie am folosit și target encoding pe interacțiuni, iar la clasificare am preferat modele de boosting care pot combina automat mai multe split-uri slabe.

4 Preprocesare

4.1 Decizii comune

Au fost aplicate câteva reguli comune pentru ambele probleme. Coloana `id` nu a fost folosită ca predictor, fiind doar identificator. Valorile lipsă din categorice au fost înlocuite cu `Missing`; numericelor li s-a aplicat imputare cu mediană când modelul cerea date complet numerice, deoarece mediana este mai robustă decât media la outlieri. Pentru modelele liniare, variabilele categorice au fost transformate prin one-hot encoding și numericile au fost standardizate, deoarece regularizarea Ridge/Lasso depinde de scara variabilelor. Pentru modelele bazate pe arbori, standardizarea nu a fost necesară, dar s-au folosit transformări care izolează distribuțiile problematice, cum ar fi binning pentru scoruri cu extreme sau indicatori pentru valori lipsă.

4.2 Preprocesarea pentru regresie

La regresie s-a evitat folosirea țintei de clasificare ca predictor. În varianta de bază, pipeline-ul a folosit one-hot encoding pentru categorice și imputare/standardizare pentru numerice, astfel încât modelele liniare să fie corect antrenate.

	feature_1	feature_2	cramers_v	chi2_p_value
2	job_title	company_size	0.0144	0.1566
1	job_title	industry	0.0144	0.0767
18	location	remote_work	0.0136	0.1303
5	job_title	skill_bracket	0.0133	0.4177
4	job_title	remote_work	0.0132	0.4374
3	job_title	location	0.0131	0.4865
12	industry	location	0.0129	0.1182
15	company_size	location	0.0126	0.2671
13	industry	remote_work	0.0121	0.4039
14	industry	skill_bracket	0.0120	0.4258
19	location	skill_bracket	0.0116	0.5163
8	education_level	location	0.0114	0.6047
11	industry	company_size	0.0112	0.6637
10	education_level	skill_bracket	0.0102	0.1039
6	education_level	industry	0.0094	0.9569

Figura 5: Heatmap Cramer's V pentru variabilele categorice și `vacation`. Graficul evidențiază asocierile utile pentru clasificare și posibilele redundanțe între categorice.

În perfecționare au fost introduse treptat transformări mai specializate: `log1p` pe salariu, target encoding cu K-Fold pentru categorice și interacțiuni, ponderi obținute din adversarial validation, pseudo-labeling pe testul privat și seed ensembling. Pentru `aggregated_score` s-au testat variante cu păstrarea valorii brute, binning în cuantile și eliminarea valorii brute. Varianta finală a păstrat ideea de binning și a combinat doar acele ajustări care au scăzut scorul public.

4.3 Preprocesarea pentru clasificare

La clasificare, `salary` a fost exclus pentru a evita leakage-ul: în testul privat nu este disponibil și ar transforma problema într-una nerealistă. S-au folosit aceleași tratamente generale pentru lipsuri și categorice, dar metricile și scopul au fost diferite. Din cauza dezechilibrului între clase, modelele au fost analizate atât prin accuracy, cât și prin precision, recall și F1 pe clase.

Pentru perfecționarea clasificării, s-au testat variante de arbori și gradient boosting: Decision Tree cu hiperparametri variați, Random Forest / ExtraTrees cu ponderare, CatBoost nativ pe categorice, LightGBM multiclass și HistGradientBoosting. În plus, s-a folosit K-Fold pentru a verifica dacă scorul local este stabil, nu doar o întâmplare a splitului.

5 Regresie: baseline și perfecționare

5.1 Baseline obligatoriu: regresie liniară

Baseline-ul de regresie a pornit de la `LinearRegression`. Apoi au fost testate regularizări Ridge, Lasso și ElasticNet, cu mai multe valori pentru hiperparametri. Această etapă a fost utilă pentru a vedea cât din problemă poate fi rezolvată prin relații aproape liniare.

Tabelul 5: Modele liniare și regularizate pentru regresie

Model	Train MSE	Test MSE	RMSE	MAE	R^2
LinearRegression baseline	$5.5605 \cdot 10^7$	$5.4936 \cdot 10^7$	7411.91	5731.12	0.9602
Ridge $\alpha = 1$	$5.5605 \cdot 10^7$	$5.4937 \cdot 10^7$	7411.96	5731.14	0.9602
Ridge $\alpha = 10$	$5.5606 \cdot 10^7$	$5.4942 \cdot 10^7$	7412.29	5731.15	0.9602
Lasso $\alpha = 0.1$	$5.5605 \cdot 10^7$	$5.4936 \cdot 10^7$	7411.89	5731.11	0.9602
Lasso $\alpha = 1.0$	$5.5605 \cdot 10^7$	$5.4935 \cdot 10^7$	7411.82	5731.04	0.9602
Lasso $\alpha = 10$	$5.5634 \cdot 10^7$	$5.4966 \cdot 10^7$	7413.90	5732.70	0.9602
ElasticNet $\alpha = 0.01, l1 = 0.75$	$5.6215 \cdot 10^7$	$5.5512 \cdot 10^7$	7450.62	5752.57	0.9598

Regularizarea nu a schimbat semnificativ rezultatul. Ridge a stabilizat coeficienții, dar nu a adus un câștig clar, ceea ce sugerează că multicolaritatea nu era principala problemă în baseline. Lasso cu **alpha=1.0** a fost marginal mai bun, dar diferența este foarte mică, deci nu poate fi considerată o îmbunătățire structurală. Concluzia a fost că baseline-ul liniar este solid pentru relațiile globale, mai ales experiență-salariu, dar nu poate surprinde suficient de bine interacțiunile dintre rol, educație, locație, experiență și distribuția diferită din testul privat.

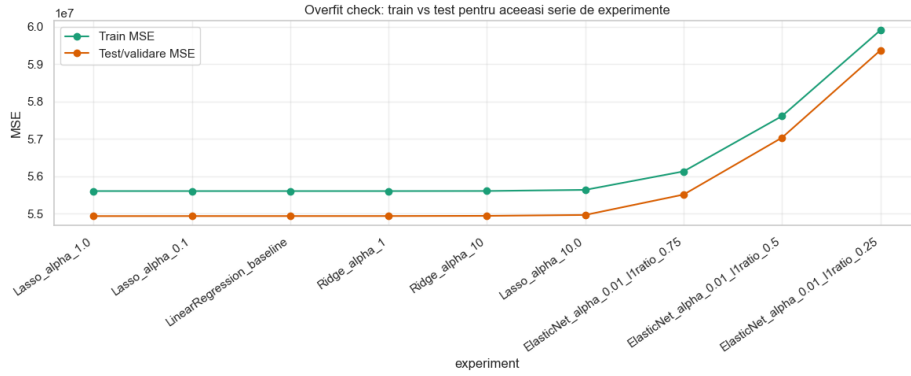


Figura 6: Curbe train/test pentru modelele de regresie locale, folosite pentru a compara baseline-ul liniar cu variantele regularizate și cu etapele mai avansate.

5.2 Adversarial validation

O observație importantă a fost că testul privat nu are exact aceeași distribuție ca trainul local. Pentru a măsura diferența, s-a antrenat un clasificator care încearcă să distingă rândurile din train de rândurile din testul privat. Scorurile obținute au fost mari: aproximativ 0.76–0.77 AUC cu Random Forest și aproximativ 0.91 AUC în experimentele CatBoost recuperate din loguri. Acest rezultat a confirmat că nu era suficient să optimizăm doar MSE-ul local.

În urma acestei analize au fost folosite ponderi adversariale în unele modele, iar ulterior pseudo-labeling, pentru ca modelul să vadă mai bine distribuția testului privat. Ideea nu a fost să memorăm leaderboard-ul, ci să corectăm diferența reală de distribuție observată în date. Important este că seed-urile au fost folosite pentru mediere și reducerea varianței, nu pentru seed hacking: nu am ales un seed izolat doar pentru că a prins un scor bun, ci am făcut ansambluri pe mai multe antrenări și am păstrat doar etapele motivate de schimbări de preprocesare/model.

5.3 Etapele de perfecționare pentru regresie

Perfecționarea regresiei a avut mai multe etape clare. Am păstrat în raport doar pașii care au schimbat direcția experimentelor sau au produs o îmbunătățire relevantă.

Tabelul 6: Jurnalul etapelor importante pentru regresie

Etapă	MSE local	RMSE	MAE	R^2	Observație
CatBoost pseudo	$2.8102 \cdot 10^7$	5301.17	4232.88	0.9797	Primul salt major: target logaritmizat și pseudo-labeling.
LGBM pseudo	$2.9070 \cdot 10^7$	5391.62	4307.28	0.9790	Model separat, mai slab local, dar util pentru blend.
CatBoost+LGBM blend	$2.71984 \cdot 10^7$	5289.99	4225.26	0.9797	Blend-ul a redus eroarea și a confirmat diversitatea modelelor.
LGBM seed ensemble	$2.8846 \cdot 10^7$	5370.85	4289.58	0.9791	Scor public mai bun: media pe seed-uri a redus varianța.
Away blend	$2.8879 \cdot 10^7$	5373.88	4291.94	0.9791	Ajustare mică în direcția modelului care generalizează mai bine pe leaderboard.
AggBins droprow 75%	$2.8935 \cdot 10^7$	5379.11	4293.96	0.9791	Transformare data-centric pentru <code>aggregated_score</code> ; scor public mai bun decât seed ensemble simplu.
Final V2 dropcert 20%	$2.8753 \cdot 10^7$	5362.21	4281.30	0.9792	Varianta finală: combinație între modelul curent cel mai bun și o variantă V2 care corectează tratamentul unor feature-uri.

	stage	train_source_for_local_metric	final_train_source	test_MSE	test_RMSE	test_MAE	test_R2	public_note
0	Stage 1 - CatBoost pseudo	train_raw	train_raw + test_raw	2.810240e+07	5301.169674	4232.878415	0.979656	intermediar; a devenit sursa pseudo
1	Stage 2 - LGBM pseudo	train_raw + private pseudo	train_raw + test_raw + private pseudo	2.906952e+07	5391.615818	4307.281481	0.978956	intermediar; LGBM separat
2	Stage 2b - CatBoost + LGBM blend	blend între Stage 1 CatBoost și Stage 2 LGBM	blend între Stage 1 CatBoost și Stage 2 LGBM	2.798396e+07	5289.986816	4225.256363	0.979742	~271.9M public pentru blendul Cat/LGBM
3	Stage 3 - LGBM seed ensemble	train_raw + private pseudo	train_raw + test_raw + private pseudo	2.884600e+07	5370.846870	4289.583180	0.979118	268.447M public
4	Stage 4 - away fata de blend Cat/LGBM	formula: LGBM_seed + alpha * (blend_slab - LGBM...)	formula: LGBM_seed + alpha * (blend_slab - LGBM...)	2.887864e+07	5373.884771	4291.938209	0.979094	268.429M public
5	Stage 5 model - aggbins droprow	train_raw + private pseudo	train_raw + test_raw + private pseudo	2.929089e+07	5412.105462	4319.674806	0.978796	
6	Stage 5 - aggbins droprow 75%	blend: 25% Stage 4 away + 75% model aggbins dr...	blend: 25% Stage 4 away + 75% model aggbins dr...	2.893485e+07	5379.111926	4293.961565	0.979054	268.241M public
7	Stage 6 model - V2 dropcert	train_raw + private pseudo	train_raw + test_raw + private pseudo	2.922210e+07	5405.747344	4312.535332	0.978846	
8	Stage 6 - V2 dropcert 20% final	blend final: 80% Stage 5 anchor + 20% V2 dropcert	blend final: 80% Stage 5 anchor + 20% V2 dropcert	2.875333e+07	5362.213622	4281.298259	0.979185	~268.1M public, final ales

Figura 7: Evoluția etapelor de perfecționare pentru regresie. Graficul/tabelul arată cum s-a trecut de la pseudo-labeling și blend-uri la seed ensembling și ajustări data-centric.

5.4 Modelul final de regresie

Modelul final nu este un singur algoritm izolat, ci un pipeline construit în jurul observației că distribuția din test este ușor diferită. Pașii principali au fost:

1. Curățarea și alinierea coloanelor între train și private test.
2. Tratarea valorilor lipsă, în special `remote_work`, ca semnal categoric.
3. Transformarea țintei prin `log1p(salary)` și revenirea la scară prin `expm1` pentru predicții.
4. Folosirea target encoding-ului cu K-Fold pentru categorice și interacțiuni importante, astfel încât encodarea să nu vadă propria țintă a rândului curent.
5. Antrenarea de modele LightGBM pe mai multe seed-uri și medierea predicțiilor.
6. Testarea unor modele diferite, CatBoost și XGBoost, ca surse de diversitate pentru blend.
7. Ajustări data-centric: binning pentru `aggregated_score`, analizarea `skills_count`, `experience_years` și `certifications`, apoi păstrarea doar a variantelor care au ajutat pe leaderboard.
8. Blend final între predicțiile care au avut comportament complementar.

Submisia finală de regresie are 16.000 de predicții, fără valori lipsă. Distribuția predicțiilor finale a fost: medie 156.985, deviație standard 35.555, minim 42.532, mediană 155.107 și maxim 311.245. Aceste valori sunt plauzibile față de distribuția salariilor din train, ceea ce a fost o verificare suplimentară înainte de submisie.

6 Clasificare: baseline și perfecționare

6.1 Baseline obligatoriu: Decision Tree

Pentru clasificare, baseline-ul cerut a fost `DecisionTreeClassifier`. Înainte de acesta a fost inclus și un `DummyClassifier` pentru a avea o bornă minimă: dacă modelul prezice mereu clasa majoritară, accuracy-ul este aproximativ 0.542. Decision Tree a depășit clar această limită și pragul de 0.6.

Tabelul 7: Baseline-uri și hiperparametri pentru clasificare

Model	Train accuracy	Accuracy test	F1 macro	F1 weighted
Dummy most frequent	0.5420	0.5420	0.1757	0.3810
DecisionTree depth 4, leaf 20	—	0.6614	—	—
DecisionTree depth 8, leaf 30	—	0.7101	—	—
DecisionTree leaf 100	0.7318	0.7256	0.6686	0.7374

Creșterea adâncimii arborelui a îmbunătățit rezultatul până la un punct, dar un arbore simplu rămâne limitat. Un arbore prea superficial pierde interacțiuni utile, iar unul prea adânc începe să memoreze combinații rare. Varianta cu `min_samples_leaf=100` a avut un echilibru mai bun, pentru că impune frunze suficient de mari. Modelele ulterioare au folosit ansambluri și boosting pentru a reduce varianța și a modela interacțiuni mai fine fără să depindă de un singur arbore.

6.2 EDA specifică pentru clasificare

Din cauza dezechilibrului claselor, accuracy-ul singur poate fi înșelător. De aceea, în analiza clasificării au fost urmărite și recall/F1 pe clase. Clasele **Medium** și **Small** sunt cele mai greu de separat, deoarece se suprapun mai mult în spațiul feature-urilor și au mai puține exemple decât **No Vacation**.

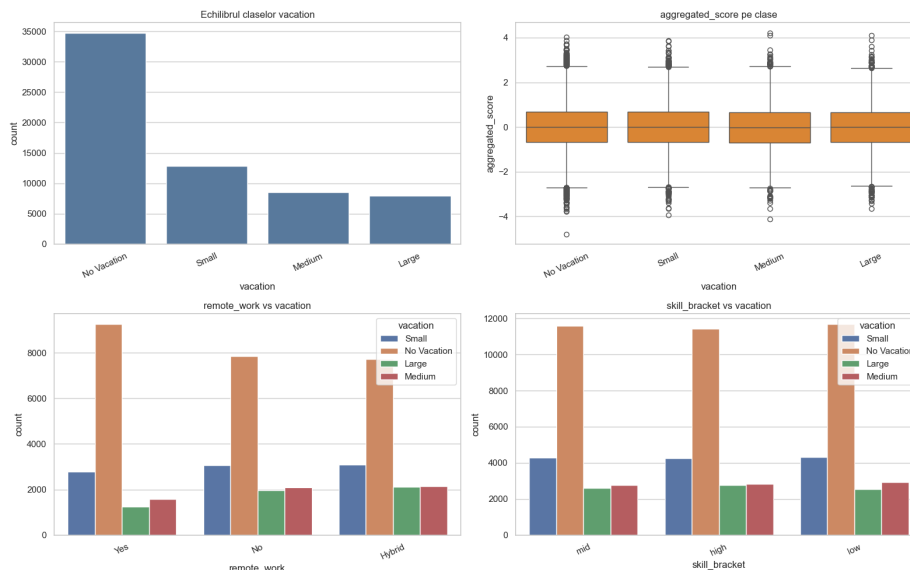


Figura 8: EDA specifică pentru clasificare: balanța claselor și relația dintre **vacation** și câteva variabile numerice/categorice relevante.

6.3 Modele încercate pentru perfecționare

Au fost testate modele cu comportament diferit. CatBoost a fost o alegere naturală deoarece gestionează bine categoricele. LightGBM și HistGradientBoosting au fost testate pentru performanță pe date tabelare, iar ExtraTrees/RandomForest pentru robustețe și diversitate.

Tabelul 8: Comparație între modelele principale de clasificare

Model	Train accuracy	Accuracy	Precision macro	Recall macro	F1 macro
ExtraTrees balanced	—	0.7432	—	—	0.6818
RandomForest balanced	—	0.7428	—	—	0.6773
CatBoost V1 FE keepraw	—	0.7591	—	—	0.6761
LightGBM multiclass	0.7903	0.7603	0.6858	0.6676	0.6759
CatBoost native	0.7647	0.7606	0.6891	0.6722	0.6801
HistGradientBoosting final	0.7965	0.7611	0.6873	0.6695	0.6777

HistGradientBoosting a fost ales ca model final după accuracy local, deși CatBoost și ExtraTrees au fost foarte apropiate sau chiar mai bune pe anumite metrice macro. Aceasta este o diferență importantă: dacă obiectivul ar fi fost exclusiv F1 macro, alegerea finală putea fi diferită. ExtraTrees ajută clasele minoritare prin randomizare și de aceea macro-F1 este competitiv, în timp ce HistGradientBoosting optimizează mai bine separarea globală și clasa majoritară. Pentru cerința temei și pentru stabilitatea locală, HistGradientBoosting a fost varianta finală de submitere.

	model	train_accuracy	accuracy	precision_macro	recall_macro	f1_macro	precision_weighted	re
0	HistGradientBoosting	0.7965	0.7611	0.6873	0.6695	0.6777	0.7536	
1	CatBoost_native	0.7647	0.7606	0.6891	0.6722	0.6801	0.7564	
2	LightGBM_multiclass	0.7903	0.7603	0.6858	0.6676	0.6759	0.7528	
3	CatBoost_V1_FE_keepraw	0.7634	0.7591	0.6862	0.6675	0.6761	0.7542	
4	CatBoost_V1_FE_dropagg	0.7639	0.7589	0.6856	0.6653	0.6745	0.7528	
5	CatBoost_V1_FE_dropcert	0.7637	0.7579	0.6843	0.6643	0.6734	0.7518	
6	ExtraTrees_balanced	0.8032	0.7432	0.6792	0.6889	0.6818	0.7655	
7	RandomForest_balanced	0.8123	0.7428	0.6756	0.6823	0.6773	0.7608	
8	DecisionTree_depthNone_leaf100	0.7318	0.7256	0.6655	0.6791	0.6686	0.7577	
9	DecisionTree_depth8_leaf30	0.7165	0.7101	0.6554	0.6693	0.6570	0.7523	
10	DecisionTree_depth4_leaf20	0.6614	0.6614	0.6030	0.5461	0.5325	0.6594	
11	Dummy_most_frequent	0.5420	0.5420	0.1355	0.2500	0.1757	0.2938	

Figura 9: Comparație între modelele de clasificare testate, de la Decision Tree și ensemble-uri de arbori până la boosting și CatBoost.

6.4 Evaluarea finală pentru clasificare

Modelul final a fost evaluat prin matrice de confuzie și metrici per clasă. Clasa **No Vacation** este recunoscută cel mai bine, iar confuziile cele mai frecvente apar între **Small** și **Medium**. Acest lucru este coerent cu natura problemei: aceste clase sunt apropiate semantic și probabil împart multe profiluri similare.

Tabelul 9: Metrici per clasă pentru HistGradientBoosting final

Clasă	Precision	Recall	F1	Support
Large	0.7772	0.7410	0.7587	1977
Medium	0.5434	0.4812	0.5104	2132
No Vacation	0.8737	0.9171	0.8949	8672
Small	0.5551	0.5387	0.5467	3219

Tabelul 10: Matricea de confuzie pentru HistGradientBoosting final

Actual / Prezis	Large	Medium	No Vacation	Small
Large	1465	376	63	73
Medium	364	1026	101	641
No Vacation	12	31	7953	676
Small	44	455	986	1734

Pentru stabilitate, modelul final a fost verificat și cu 3-Fold cross-validation. Accuracy-ul pe folduri a fost 0.7600, 0.7562 și 0.7615, cu medie aproximativ 0.7592. F1 macro mediu a fost aproximativ 0.6702. Variația mică între folduri sugerează că modelul nu depinde excesiv de un singur split. Matricea de confuzie arată totuși limita principală: **No Vacation** este

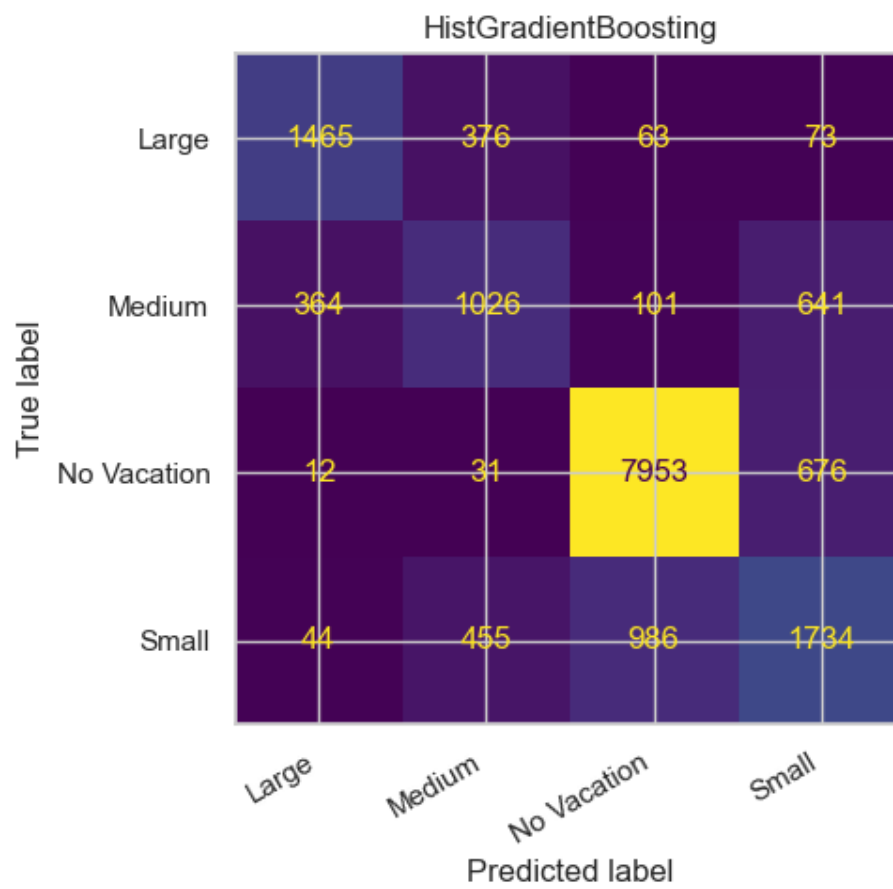


Figura 10: Matricea de confuzie pentru modelul final de clasificare. Cele mai dificile separări apar între clasele **Small** și **Medium**.

prezică cel mai bine, iar **Medium** are recall mai slab deoarece se confundă des cu **Small**. Aceasta este consecința directă a dezechilibrului de clase și a suprapunerii dintre clasele intermediare.

7 Rezultate Kaggle și fișiere finale

Pentru regresie, cel mai bun rezultat a venit după combinația dintre seed ensembling, transformări data-centric și blend-ul final. Scorul public a coborât de la variante de aproximativ 273M la aproximativ 268M. Diferențele de la final au devenit mici, ceea ce indică faptul că partea de modelare era deja aproape saturată, iar câștigurile rămase veneau mai ales din procesarea datelor și alegerea predicțiilor de combinat.

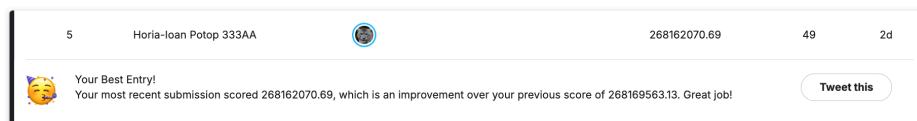


Figura 11: Screenshot cu scorul final de regresie pe Kaggle.

Pentru clasificare, submisia finală generată este `submission_classification_final.csv`. Distribuția predicțiilor pe testul privat a fost verificată pentru a nu produce un colaps într-o singură clasă: aproximativ 34.08% **No Vacation**, 28.26% **Small**, 21.23% **Medium** și 16.43% **Large**.

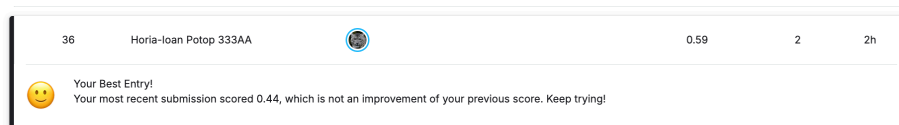


Figura 12: Verificarea finală pentru submisia de clasificare / distribuția predicțiilor generate.

Fișierele principale de predare sunt:

- `main.ipynb`: notebook-ul complet, cu EDA, preprocesare, antrenare și evaluare.
- `submission_regression_final.csv`: submisia finală pentru regresie.
- `submission_classification_final.csv`: submisia finală pentru clasificare.
- `regression_csv_selected/`: CSV-uri reprezentative pentru baseline și etapele importante de perfecționare.
- `classification_csv_selected/`: rezultate, rapoarte și submisii relevante pentru clasificare.

8 Concluzii

Pentru regresie, cele mai mari câștiguri nu au venit dintr-un singur hiperparametru, ci din înțelegerea distribuției datelor. Adversarial validation a arătat că testul privat diferă de train, pseudo-labeling a introdus informație despre această distribuție, iar seed ensembling-ul LightGBM a redus variația. Experimentele finale cu `aggregated_score`, binning și tratamentul unor variabile au arătat că, după ce modelul este deja puternic, micile decizii de preprocesare pot conta mai mult decât schimbarea algoritmului.

Pentru clasificare, baseline-ul Decision Tree a depășit pragurile obligatorii, dar modelele de boosting au oferit rezultate mai bune. HistGradientBoosting a avut cel mai bun accuracy local, CatBoost a fost foarte apropiat și mai bun pe unele metrice macro, iar ExtraTrees a

arătat că un model mai randomizat poate ajuta clasele minoritare. Matricea de confuzie confirmă că problema rămasă este separarea claselor **Small** și **Medium**.

Dacă aş continua tema, următorii paşi ar fi: o optimizare mai atentă pentru F1 macro la clasificare, calibrare de probabilităţi pentru clasele minoritare, o validare adversarială separată pentru clasificare şi un stacked ensemble controlat pentru regresie, nu doar blend manual între CSV-uri. Totuşi, pentru forma finală a temei, soluţia include toate cerinţele de baseline, evaluare, comparaţie şi justificare a experimentelor.